Praxis Final Report

Tuan Son Truong, 48211791

Table of Contents

Abstract 3

Introduction 3

Literature Survey 3

Approach 4

Result 8

Concluding Remark 10

Appendix 11

References 14

Abstract

Training computers to recognize human motion has been one of the many applications of machine learning. However, a large part of the research has been focused on computer vision or the use of multiple sensors on different body parts to detect simple human motion. This praxis explores a different approach to motion recognition using only smartphone sensors. The objective is to classify motions into two categories: bringing the phone to eye level (yes motion) or other movements (no motion). Accelerometer and gyroscope data were collected over 5 seconds intervals to train a 12 layers Convolutional Neural Network (CNN) to perform this task. The results demonstrate the model's effectiveness in distinguishing intentional motions from background activities with an accuracy of 84.67% ± 3.32%.

Introduction

In today's world, smartphones have become an integral part of daily life. According to a 2023 survey by Reviews.org, the average American checks their phone 144 times per day. This frequent interaction has driven smartphone designs to prioritize convenience and ease of use. One such convenience that often goes unnoticed by the user but is sorely missed when absent is the smartphone's ability to recognize when the user wants to interact with it. This capability is not only necessary for the user’s experience but also conserves power by ensuring the display turns on only when needed.

Given the high frequency of phone usage, it is crucial for smartphones to reliably distinguish between the intentional motion of being brought to a user's face and random background movements. Typically, users raise their phones to eye level when interacting with them, making this motion a clear indicator to activate the display. However, as smartphones frequently experience various movements it becomes essential to differentiate these background motions from the deliberate action of bringing the phone to the face.

To address this challenge, my praxis utilizes machine learning to train a model capable of categorizing motion into two distinct categories: movements where the phone is brought to the user's face and those that are unrelated. The primary goal is to accurately identify intentional interactions while filtering out random background motion, ensuring the phone activates only when needed by the user.

Literature Survey

Training machine learning models to recognize human motion is a field known as Human Activity Recognition (HAR). Previous HAR studies, like the one done by Vidya and Sasikumar, have primarily focused on the categorization of a wide variety of simple motions such as sitting, walking, running, etc. To fully encapsulate these movements, Vidya and Sasikumar record their

data using multiple sensors attached to different parts of the person's body and a smartphone. Because the goal of the praxis is to train the phone to do binary categorization to see if a person is using their phone or not, I will only utilize my phone to capture my data.

Modern smartphones come equipped with a wide range of sensors. However, utilizing multiple sensors simultaneously can significantly increase energy consumption, which is a critical concern for low-power devices like smartphones. Given the goal of this praxis, to continuously monitor whether the user intends to use the phone, it is essential to balance sensor usage for accuracy while minimizing energy consumption. According to studies by Shoaib et al., the accelerometer and gyroscope are well-suited to play primary roles in recognizing simple motions. Although the study has shown that combining these sensors may not always yield a noticeable improvement in overall HAR performance. However, I will still use both in tandem, as each excels in capturing distinct aspects of a more complicated movement, such as bringing a phone to eye level.

Studies on Human Activity Recognition (HAR) often utilize a wide range of machine learning models. One such study done by Logacjov et al. seeks to compare the different machine learning models to find which one has the best performance. According to the study, Support Vector Machine (SVM) slightly outperformed other models, achieving the highest F1 score, a combination of precision and recall. However, according to a different study by Ramamurthy and Roy, surveying the trends of machine learning in HAR, SVM relies heavily on feature engineering and uses shallow learning techniques. These characteristics make them less suitable for tasks involving complex motions, such as bringing a phone to eye level. In contrast, Ramamurthy and Roy observed that deep learning methods, such as CNN, excel at recognizing complex motion without the need for extensive feature engineering. Therefore, CNN is a more compatible choice for achieving the praxis' goal.

Approach

Data Collection:

One significant challenge in training a machine model to recognize human motion is collecting data that allows the model to generalize effectively across individuals. As Ramamurthy and Roy point out, the probability distribution of accelerometer data is highly user dependent. As a result, a model trained on data from one individual, even if it performs well for that person, may not yield reliable results for others. Given this challenge, along with the absence of a publicly available dataset specifically for the motion of bringing a phone to eye level, I opted to record my own data. This approach differs from other studies done using smartphones for HAR as it enables me to personalize the CNN model and provides the flexibility to expand the dataset if required.

To maintain class balance in my dataset, I categorized the data into two classes for binary classification: motions involving bringing the phone to eye level (yes data) and motions that don't (no data). Given the diverse ways smartphones are used daily, the dataset is also needed to capture a variety of realistic scenarios. As a result, I will divide my yes and no data into smaller scenarios, with each containing its own sub-scenario, with 20 recordings per sub-scenario—half performed with the right hand and the other half with the left. The data collection will be done through an iPhone 14 plus using the MATLAB mobile app sensor measurement feature. The phone will measure its acceleration and rotation in the x, y, and z directions using its built-in accelerometer and gyroscope with a frequency of 100Hz.

The "Yes Data" is divided into four main scenarios: **Lying**, **Sitting**, **Standing**, and **Stairs**, with each scenario containing sub-scenarios—220 recording total.

1. **Lying**:
    - Bringing the phone to eye level starting from near the head.
    - Bringing the phone to eye level starting from near the side.
2. **Sitting**:
    - Raising the phone when it face-up on a table.
    - Flipping the phone when it face-down on a table.
3. **Standing**:
    - Bringing the phone to eye level while stationary.
    - Bringing the phone to eye level while walking.
    - Bringing the phone to eye level while running.
4. **Stairs**:
    - Bringing the phone to eye level while walking up the stairs.
    - Bringing the phone to eye level while running up the stairs.
    - Bringing the phone to eye level while walking down the stairs.
    - Bringing the phone to eye level while running down the stairs.

Because the "No data" encompasses a wide variety of motions, there are many ways to record it. To avoid having more "No Data" then "Yes Data" and creating bias during training, I prioritized no motions that start similarly to the yes motion, such as grabbing the phone from a table, bringing it to the ear, or raising it at an angle away from eye level.

The "No Data" is divided into three main scenarios: **Grabbing**, **Standing**, and **Stairs**, with each scenario containing sub-scenarios—180 recording total.

1. **Grabbing**:
    - Grabbing the phone when it is face-up on a table
    - Grabbing the phone when it is face-down on a table
2. **Standing**:
    - Performing the motion while stationary.
    - Performing the motion while walking.

- Performing the motion while running.
- For each motion:
  - First 10 recordings: Bringing the phone to the right and left ear
  - Last 10 recordings: Bringing the phone to the face at an angle away from eye level with right and left hand

3. **Stairs**:
   - Performing the motion while walking up the stairs.
   - Performing the motion while running up the stairs.
   - Performing the motion while walking down the stairs.
   - Performing the motion while running down the stairs.
   - For each motion:
     - First 10 recordings: Bringing the phone to the right and left ear
     - Last 10 recordings: Bringing the phone to the face at an angle away from eye level with right and left hand

Data preprocessing:

As for the testing data, I decided to record an additional 40 data sets, evenly split between "yes" and "no" data. Unlike the training and validation datasets, the testing data includes more varied and non-uniform background motions, such as transitioning from walking to running while bringing the phone to eye level, turning to one side while lying down, or walking and running down a spiral stair case. These scenarios introduce additional noise compared to the training and validation data, providing a more challenging set of samples for the CNN to classify accurately.

Before the data can be used in in the CNN, it must first undergo preprocessing. This process involves standardizing the size of all recordings. This involves standardizing the size of all recordings. Since most recordings slightly exceed 5 seconds, the data is trimmed to 500 samples. The accelerometer and gyroscope data are then combined into a single matrix with dimensions of 500x6, where the first three columns represent accelerometer data, and the last three columns represent gyroscope data. Because accelerometer readings tend to have larger magnitudes than gyroscope readings, this imbalance could lead to faster convergence for the accelerometer data compared to the gyroscope data. To prevent this, all data is scaled to the range of $[-1,1]$, ensuring balanced learning across features. Finally, the data is then assigned a label: "1" if the motion involves bringing the phone to eye level, and "0" if it does not. To achieve a 1:4 ratio of validation to training data, the 1st, 10th, 11th, and 20th recordings are allocated to the validation dataset, while the remaining recordings are used for training.

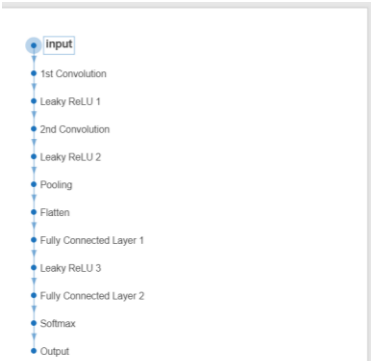
Implementation of CNN:

In CNN, stacking convolutional layers enables the network to capture low-level features in the beginning layer and high-level features in the later layer. After each convolutional layer, Logacjov et al. recommend using a ReLU activation function to introduce non-linearity, enabling

the network to learn complex patterns. However, since the accelerometer and gyroscope data contain both positive and negative values across the x, y, and z axes, using a standard ReLU could zero out all negative values, potentially discarding valuable information. To mitigate this, I chose to use a leaky ReLU activation function, which allows negative values to contribute to the model by scaling them with a small factor, ensuring no negative data is lost. Since a leaky ReLU activation is employed, He Normal initialization is used to initialize weights before the convolutional layers to ensure stable training by mitigating the risk of vanishing or exploding gradients. To further reduce the data for classification, an average pooling is used to preserve the general pattern and smooth out any noise. Alter pooling the result is then flattened and connected to a fully connected layer. The output of the layer is then passed into a SoftMax layer that converts it to a probability distribution to be used for binary classification.

According to Logacjov et al., the optimal number of convolutional layers is four. However, given the smaller size of my personalized dataset, I found that using two convolutional layers is sufficient to achieve high training and validation accuracy while maintaining low training and validation time. Both convolution filters use a 5x1 filter size with a stride of 1 and no padding. The first convolutional layer recieve an input matrix of 500×6, applying 16 filters to produce 16 feature maps of size 495×6. These outputs are then passed to the second convolutional layer, which uses 32 filters to generate 32 feature maps of size 490×6 times. For my average pooling layer, I have selected a filter size of 2x1 with a horizontal stride of 1 and a vertical stride of 2. This reduces the number of rows in the feature maps by half, transforming the 490x6 feature map into a 245x6 feature map. After pooling, the output is flattened into a 1x1470 vector. The flattened vector is then connected to a fully connected layer with 64 neurons, whose output is passed to another fully connected layer with 2 neurons. These two neurons represent the possible classifications of the motion. Finally, a SoftMax layer is applied to output the probability for the binary classification.

Figure 1: CNN architecture



LAYER INFORMATION

	Name	Type	Activations	Learnable Sizes	State Sizes
1	input 500×6×1 images with 'zerocenter' norma...	Image Input	500(S) × 6(S) × 1(C) × 1(B)	-	-
2	1st Convolution 16 5×1×1 convolutions with stride [1 1] a...	2-D Convolution	500(S) × 6(S) × 16(C) × 1(B)	Weig... 5 × 1 × 1 ... Bias 1 × 1 × 16	-
3	Leaky ReLU 1 Leaky ReLU with scale 0.1	Leaky ReLU	500(S) × 6(S) × 16(C) × 1(B)	-	-
4	2nd Convolution 32 5×1×16 convolutions with stride [1 1] ...	2-D Convolution	500(S) × 6(S) × 32(C) × 1(B)	Weig... 5 × 1 × 16... Bias 1 × 1 × 32	-
5	Leaky ReLU 2 Leaky ReLU with scale 0.1	Leaky ReLU	500(S) × 6(S) × 32(C) × 1(B)	-	-
6	Pooling 2×1 average pooling with stride [2 1] an...	2-D Average Pooling	250(S) × 6(S) × 32(C) × 1(B)	-	-
7	Flatten Flatten	Flatten	48000(C) × 1(B)	-	-
8	Fully Connected Layer 1 64 fully connected layer	Fully Connected	64(C) × 1(B)	Weights 64 × 48000 Bias 64 × 1	-
9	Leaky ReLU 3 Leaky ReLU with scale 0.01	Leaky ReLU	64(C) × 1(B)	-	-
10	Fully Connected Layer 2 2 fully connected layer	Fully Connected	2(C) × 1(B)	Weights 2 × 64 Bias 2 × 1	-
11	Softmax softmax	Softmax	2(C) × 1(B)	-	-
12	Output crossentropyex with classes '0' and '1'	Classification Output	2(C) × 1(B)	-	-

For backpropagation and weight updates, I selected stochastic gradient descent with a step size of 0.01 and 25 epochs. The choice of 25 epochs was based on observing that the training accuracy reaches 100%, while the validation accuracy stabilizes at 97.5%. As shown in the table below, increasing the number of epochs beyond 25 does not result in any significant improvement and only increases the training time for the CNN.

Table 1: Training Progress for 100 epochs

```
Training on single CPU.
Initializing input data normalization.
|=========================================================================================================================|
|  Epoch  |  Iteration  |  Time Elapsed  |  Mini-batch  |  Validation  |  Mini-batch  |  Validation  |  Base Learning  |
|         |             |   (hh:mm:ss)   |   Accuracy   |   Accuracy   |     Loss     |     Loss     |      Rate       |
|=========================================================================================================================|
|       1 |           1 |       00:00:01 |       50.00% |       87.50% |       0.7542 |       0.3999 |          0.0100 |
|      25 |          50 |       00:00:16 |      100.00% |       97.50% |       0.0654 |       0.1275 |          0.0100 |
|      50 |         100 |       00:00:31 |      100.00% |       97.50% |       0.0236 |       0.0988 |          0.0100 |
|      75 |         150 |       00:00:47 |      100.00% |       97.50% |       0.0127 |       0.0955 |          0.0100 |
|     100 |         200 |       00:01:02 |      100.00% |       97.50% |       0.0082 |       0.0956 |          0.0100 |
|=========================================================================================================================|
Training finished: Max epochs completed.
```

Result

Due to the random initialization of weights of the CNN, each training session results in a slightly different model with varying levels of accuracy. To comprehensively evaluate the performance of my CNN, I will run the training process 20 times and calculate the average training, validation, and testing accuracies. However, to accurately gauge the effectiveness of my CNN model at binary classification, I will also compute the precision, recall, and F1 score of the model for the testing data.

As shown in the table, the training accuracy for all trials consistently reaches 100%. However, the validation accuracy averages around 97.63%, suggesting that the CNN is slightly overfitting and performs slightly worse on unseen data. Moreover, the testing data achieves an average accuracy of only 84.67%. The model's average precision of 83.89% indicates that it reliably identifies the motion of picking up the phone. Additionally, the average recall of 88.44% demonstrates that the model successfully captures most instances of the picking-up phone motion, with relatively few missed cases. The lower testing accuracy, relative to the training and validation accuracy, may be attributed to increased noise in the testing data compared to the training and validation datasets.

Table 2: CNN performance over 20 trials

Trial	Training Accuracy (%)	Validaiton Accuracy (%)	Testing Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
1	100	97.50	83.33	82.35	87.50	84.85
2	100	97.50	86.67	87.50	87.50	87.50
3	100	97.50	90.00	88.24	93.75	90.91
4	100	96.25	90.00	88.24	93.75	90.91
5	100	96.25	83.33	82.35	87.50	84.85
6	100	98.75	83.33	82.35	87.50	84.85
7	100	98.75	80.00	81.25	81.25	82.25
8	100	97.50	90.00	93.33	87.50	90.32
9	100	98.75	83.33	82.35	87.50	84.85
10	100	98.75	86.67	83.33	93.75	88.24
11	100	96.25	80.00	75.00	93.75	83.33
12	100	98.75	86.67	83.33	93.75	88.24
13	100	93.75	86.67	83.33	93.75	88.24
14	100	98.75	83.33	82.35	87.50	84.85
15	100	98.75	83.33	82.35	87.50	84.85
16	100	98.75	80.00	85.71	75.00	80.00
17	100	97.50	83.33	82.35	87.50	84.85
18	100	97.50	86.67	83.33	93.75	88.24
19	100	97.50	80.00	81.25	81.25	81.25
20	100	97.50	86.67	87.50	87.50	87.50

Table 3: Average CNN results

Average Training Accuracy (%)	100 ± 0
Average Validaiton Accuracy (%)	97.63 ± 1.28
Average Testing Accuracy (%)	84.67 ± 3.32
Average Precision (%)	83.89 ± 5.08
Average Recall (%)	88.44 ± 5.08
Average F1 Score (%)	86.04 ± 3.08

Compared to the results of Logacjov et al., who evaluated various machine learning models, our CNN performs on par with their CNN and other models. However, our approach uses fewer convolutional layers (2 layers instead of 4) and fewer training epochs (25 epochs instead of 80). Additionally, the standard deviation of our precision, recall, and F1-score is significantly smaller, indicating that our CNN models are more consistent, with each performing similarly. This difference could be attributed to the distinct goals of each study. Logacjov et al. focused on multi-class classification of simpler motions, which inherently introduces greater variability and higher standard deviations due to a larger set of possible classifications. In contrast, our work emphasizes the binary classification of a more complex motion, leading to reduced variability in model performance.

Table 4: Result of the Logacjov et al. study

	k-NN	SVM	RF	XGB	BiLSTM	CNN	mCNN
Recall	0.75 ± 0.26	0.79 ± 0.22	0.73 ± 0.31	0.78 ± 0.24	0.77 ± 0.21	0.79 ± 0.20	0.79 ± 0.20
Precision	0.83 ± 0.15	0.85 ± 0.13	0.83 ± 0.14	0.84 ± 0.15	0.81 ± 0.17	0.82 ± 0.17	0.82 ± 0.15
F1-score	0.78 ± 0.22	0.81 ± 0.18	0.76 ± 0.25	0.80 ± 0.20	0.79 ± 0.19	0.80 ± 0.19	0.80 ± 0.18

Concluding Remark

In this praxis, we explored the approach of using CNN for human activity recognition by focusing on the binary classification of a complex motion: bringing a phone to eye level. Unlike traditional methods that rely on multiple sensors, our approach utilizes only the accelerometer and gyroscope data from a single sensor, our smartphone. Although the CNN model performed worse on the testing data compared to the validation and testing data, the model still tested with an accuracy of 84.67%, along with its average precision of 83.89% and recall of 88.44%. As a result, it demonstrates that our CNN model has the ability to distinguish intentional phone motions from background activity.

To better improve the reliability of my CNN model and align the testing accuracy with the training and validation accuracy, several improvements can be made. First, incorporating noisier data, such as transitions between multiple background motions or scenarios like walking up a spiral staircase instead of a straight one, could help the model generalize better. Additionally, training and validation data can be diversified by including different ways people bring their phones to their faces, such as varying angles of motion. To ensure that the model performs well regardless of the user, data should be collected from multiple individuals. Lastly, for practical application, the duration of input data needs to be reduced. Currently, my CNN requires 5 seconds of data for classification, which delays the phone's ability to turn on instantly when raised. Reducing this to 1 second would enable faster response times, though it would require additional effort to process and optimize the data.

Appendix

Matlab Code for CNN:

```
clear;
% Loading the training data
trainingDataFolder = "C:\Users\Kevin\Desktop\Skewl\Praxis\TrainingData";
trainingData = zeros(500, 6, 1, 320);
trainingDataLabel = zeros(1, 320);
for i = 1:320
    % Construct full file path to each .mat file
    filePath = fullfile(trainingDataFolder, sprintf("%d.mat", i));

    % Load the .mat file
    matFile = load(filePath);

    % Store the data into the 3D array
    trainingData(:, :, 1, i) = matFile.X;
    trainingDataLabel(1,i) = matFile.label;
end
trainingDataLabel = categorical(trainingDataLabel);  % Convert training labels

% Loading the validation data
validationDataFolder = "C:\Users\Kevin\Desktop\Skewl\Praxis\ValidationData";
validationData = zeros(500, 6, 1, 80);
validationDataLabel = zeros(1,80);
for i = 1:80
    % Construct full file path to each .mat file
    filePath = fullfile(validationDataFolder, sprintf("%d.mat", i));

    % Load the .mat file
    matFile = load(filePath);

    % Store the data into the 3D array
    validationData(:, :, 1, i) = matFile.X;
    validationDataLabel(1,i) = matFile.label;
end
validationDataLabel = categorical(validationDataLabel);  % Convert validation
labels

% Clean up the workspace
clear matFile filePath i trainingDataFolder validationDataFolder

% Building the CNN layers
CNNsetting = [
    % Input layer
```

```
    imageInputLayer([500 6 1], "Name", "input");

    % Convolution layer 1
    convolution2dLayer([5 1], 16, "Padding", "same", "Name", "1st Convolution",
"WeightsInitializer", "he");
    leakyReluLayer(0.1,"Name","Leaky ReLU 1");

    % Convolution layer 2
    convolution2dLayer([5 1], 32, "Padding", "same", "Name", "2nd Convolution",
"WeightsInitializer", "he");
    leakyReluLayer(0.1,"Name","Leaky ReLU 2");

    % Pooling layer
    averagePooling2dLayer([2 1], "Stride", [2 1], "Name", "Pooling");

    % Flatten Layer
    flattenLayer("Name", "Flatten");

    % Fully connected layer 1
    fullyConnectedLayer(64, "Name", "Fully Connected Layer 1",
"WeightsInitializer", "he");
    leakyReluLayer(0.01, "Name", "Leaky ReLU 3");

    % Fully connected layer 2 and output
    fullyConnectedLayer(2, "Name", "Fully Connected Layer 2");
    softmaxLayer("Name", "Softmax")
    classificationLayer("Name", "Output");
];

% Building the testing setting
trainingSetting = trainingOptions("sgdm", "InitialLearnRate", 0.01, "Momentum",
0 , "MaxEpochs", 25, "ValidationData", {validationData,validationDataLabel},
"Plots", "training-progress");

% Passing in the training and validation data
training = trainNetwork(trainingData, trainingDataLabel, CNNsetting,
trainingSetting);
analyzeNetwork(training)

% Loading in the testing data
testingDataFolder = "C:\Users\Kevin\Desktop\Skewl\Praxis\TestingData";
file = dir(fullfile(testingDataFolder, "*.mat"));
testingData = zeros(500, 6, 1, 30);
testingDataLabel = zeros(1,30);
for i = 1:30
    % Construct full file path to each .mat file
```

```
    filePath = fullfile(testingDataFolder, file(i).name);

    matFile = load(filePath);

    testingData(:, :, 1, i) = matFile.X;
    testingDataLabel(1,i) = matFile.label;

end
testingDataLabel = categorical(testingDataLabel);

% Clean up the workspace
clear matFile filePath file i testingDataFolder

predictedLabel = classify(training,testingData);
accuracy = sum(predictedLabel == testingDataLabel') / numel(testingDataLabel) *
100;
disp(['Testing Accuracy: ', num2str(accuracy), '%']);

% Confusion Matrix
confusionMatrix = confusionmat(testingDataLabel, predictedLabel);

% Extract values from confusion matrix
truePositive = confusionMatrix(2, 2);       % Correctly predicted '1'
falsePositive = confusionMatrix(1, 2);      % Predicted '1', but actual '0'
falseNegative = confusionMatrix(2, 1);      % Predicted '0', but actual '1'
trueNegative = confusionMatrix(1, 1);       % Correctly predicted '0'

% Calculate and display precision, recall, and F1-Score
precision = truePositive / (truePositive + falsePositive);
recall = truePositive / (truePositive + falseNegative);
f1Score = 2 * (precision * recall) / (precision + recall);

fprintf('Precision: %.2f%%\n', precision * 100);
fprintf('Recall: %.2f%%\n', recall * 100);
fprintf('F1-Score: %.2f%%\n', f1Score * 100);
```

References

[1] A. Logacjov, K. Bach, A. Kongsvold, H. B. Bardstu, and P. J. Mork, "HARTH: A Human Activity Recognition Dataset for Machine Learning," *Sensors (Basel, Switzerland)*, vol. 21, no. 23, pp. 7853-, 2021, doi: 10.3390/s21237853.

[2] B. Vidya and S. P, "Wearable multi-sensor data fusion approach for human activity recognition using machine learning algorithms," *Sensors and actuators. A. Physical.*, vol. 341, pp. 113557-, 2022, doi: 10.1016/j.sna.2022.113557.

[3] S. Ramasamy Ramamurthy and N. Roy, "Recent trends in machine learning for human activity recognition—A survey," *Wiley interdisciplinary reviews. Data mining and knowledge discovery*, vol. 8, no. 4, pp. e1254-n/a, 2018, doi: 10.1002/widm.1254.

[4] M. Shoaib, S. Bosch, O. D. Incel, H. Scholten, and P. J. M. Havinga, "Fusion of Smartphone Motion Sensors for Physical Activity Recognition," *Sensors (Basel, Switzerland)*, vol. 14, no. 6, pp. 10146–10176, 2014, doi: 10.3390/s140610146.

[5] *Reviews.org*, "Cell phone addiction statistics: The definitive guide 2024," *Reviews.org*. [Online]. Available: https://www.reviews.org/mobile/cell-phone-addiction/.